

AIAgentX Low-Level Production Implementation Blueprint

Reference implementation blueprint

This is a detailed build specification based on the conceptual AIAgentX documentation. It makes deliberate, replaceable v1 engineering decisions because no application source exists in the repository.

Status: Proposed reference implementation

Audience: Engineers, platform operators, security reviewers, and technical leads

Scope: A production-ready implementation plan for the conceptual AIAgentX runtime

Evidence boundary: This repository contains no application source code. All technology choices and interfaces below are recommended implementation decisions, not reverse-engineered facts.

1. Decision record and implementation target

Build AIAgentX as a multi-tenant Python service that exposes a versioned REST API and server-sent event stream. The control plane persists agent definitions, run state, audit records, and durable memory in PostgreSQL. Stateless API pods submit runs to Redis-backed workers. Workers execute model and tool steps under policy, stream progress through an event bus, and write a durable result.

The first production release should optimize for safety, observability, deterministic failure handling, and a small API surface. Do not ship autonomous network or shell tooling by default. A tenant must explicitly enable a tool, and every tool execution must pass a policy decision.

Recommended stack

Area	Decision	Rationale
Runtime	Python 3.12, FastAPI, Uvicorn	Async API, mature validation, simple OpenAPI generation.
Validation	Pydantic v2	Explicit request, response, and configuration contracts.
Persistence	PostgreSQL 16, SQLAlchemy 2, Alembic	ACID run state, row-level authorization, migrations, and JSON metadata.
Retrieval memory	PostgreSQL with pgvector	Keeps durable memory, metadata filters, and tenant isolation in one datastore for v1.
Cache and queue	Redis 7 plus Dramatiq	Rate limits, idempotency locks, cancellation signals, and background work.
Model access	Provider adapter layer	Enables a configured primary provider and controlled fallback without leaking provider SDKs into domain code.
API auth	OIDC JWT for users; hashed scoped API keys for services	Supports browser users and service-to-service clients.
Observability	OpenTelemetry, structured JSON logs, Prometheus metrics	Trace a run across API, worker, provider, and tool calls.
Delivery	OCI image, Kubernetes, managed PostgreSQL and Redis	Horizontal scaling, isolated secrets, rolling release controls.

Fixed v1 boundaries

- The public API is /v1; breaking changes require a new version.
- Each run is tenant-scoped and has a durable state machine.
- Tool invocation is deny-by-default and runs through one policy gateway.
- Durable memory is opt-in per agent. Ephemeral memory expires with the run.
- Telemetry uses an outbox table and never blocks a client result.
- Provider fallback is permitted only before an irreversible tool effect, unless the operation has an idempotency key.

2. Repository layout and ownership boundaries

Use a layered package layout. API handlers must not talk directly to provider SDKs, Redis, or SQLAlchemy sessions. The application layer owns use cases; infrastructure implements ports.

```
aiagentx/  
  app/  
    main.py           # FastAPI factory and lifespan  
    api/v1/           # HTTP handlers, schemas, dependencies  
    domain/           # Entities, value objects, state transitions  
    application/      # Use cases and ports  
    infrastructure/  
      db/             # SQLAlchemy models, repositories, migrations  
      cache/          # Redis rate limits, locks, cancellation  
      queue/          # Dramatiq actors and worker bootstrapping  
      providers/      # OpenAI, Anthropic, or other model adapters  
      tools/          # Registered tool implementations and sandbox clients  
      observability/  # OTel, logging, metrics, outbox publishing  
      workers/        # Run executor and recovery sweeper  
      settings.py     # Typed environment configuration  
  tests/  
    unit/ integration/ contract/ e2e/ load/  
  deploy/
```

```
helm/ terraform/ dashboards/ alerts/
docs/
```

Module contracts

```
class ModelProvider(Protocol):
    async def complete(self, request: ModelRequest) -> ModelResponse: ...

class Tool(Protocol):
    name: str
    input_schema: dict[str, object]
    async def invoke(self, context: ToolContext, arguments: dict[str, object]) ->
        ToolResult: ...

class MemoryStore(Protocol):
    async def retrieve(self, query: MemoryQuery) -> list[MemoryRecord]: ...
    async def append(self, write: MemoryWrite) -> None: ...

class RunRepository(Protocol):
    async def claim(self, run_id: UUID, worker_id: str, lease_seconds: int) -> Run: ...
    async def transition(self, run_id: UUID, to_state: RunState, event: RunEvent) -> Run:
        ...
```

api depends on application; application depends on domain and ports; infrastructure implements ports. Enforce this direction in an import-lint test.

3. Core domain model and run state machine

Aggregate definitions

Aggregate	Required fields	Invariants
Tenant	id, slug, plan, status	A suspended tenant cannot create runs.
AgentDefinition	id, tenant_id, name, system_prompt, model_policy, memory_mode, version	Published definition is immutable; update creates a new version.
ToolGrant	agent_version_id, tool_name, policy_json	Tool must exist in registry and be allowlisted for the tenant.
Run	id, tenant_id, agent_version_id, state, input_json, budget, idempotency_key	One active run per idempotency key and tenant.
RunStep	run_id, sequence, kind, state, input_redacted, output_redacted	Sequence is monotonic; terminal steps cannot change.
MemoryRecord	tenant_id, agent_id, scope, content_ciphertext, embedding, expires_at	Tenant filter is mandatory for every retrieval.

States

```
queued -> running -> awaiting_approval -> running -> succeeded
              -> cancelled
              -> timed_out
              -> failed

queued -> cancelled
running -> retry_scheduled -> queued
```

Only the worker transitions running states. The API can request cancellation by atomically setting `cancel_requested_at`; workers must check before model calls, before each tool call, and while polling a long-running tool.

Transition rules

- queued -> running: worker obtains a lease using `SELECT ... FOR UPDATE SKIP LOCKED` or an atomic repository claim.
- running -> awaiting_approval: a policy requires human approval for a proposed effectful tool action.
- awaiting_approval -> running: approval token is valid, unexpired, bound to the tenant and run step.
- running -> retry_scheduled: failure is classified transient, retry budget remains, and no unprotected side effect occurred.
- Any non-terminal state -> cancelled: cancellation request wins before the next new side effect.
- Terminal states are immutable. A post-run redaction creates a new artifact revision rather than changing the raw audit record.

4. HTTP API and event contract

Use JSON over HTTPS. All mutating endpoints require Idempotency-Key; accept UUID or 32 to 128 character opaque strings. Return RFC 7807-compatible errors with type, title, status, detail, request_id, and a stable code.

Endpoint	Auth scope	Behavior
POST /v1/agents	agents:write	Creates draft agent definition version 1.
POST /v1/agents/{id}/publish	agents:write	Validates tool grants and publishes the current draft version.
POST /v1/agents/{id}/runs	runs:write	Creates a queued run and returns 202 Accepted.
GET /v1/runs/{id}	runs:read	Returns durable state, redacted output, and usage summary.
GET /v1/runs/{id}/events	runs:read	SSE stream with Last-Event-ID resume support.
POST /v1/runs/{id}/cancel	runs:write	Requests cancellation; returns current run state.
POST /v1/approvals/{id}	runs:approve	Approves or denies an awaiting tool action.
GET /healthz	none	Liveness only; no dependency calls.
GET /readyz	internal	Verifies database, Redis, and migration version.

Run request and response

```
POST /v1/agents/ag_123/runs
Idempotency-Key: 8940a0e0-3fc2-481d-8b81-23aa7865fc31
```

```
{
  "input": {"question": "Summarize this release"},
  "session_id": "s_456",
  "metadata": {"source": "web"},
  "limits": {"max_steps": 12, "max_cost_usd": "0.25", "timeout_seconds": 90}
}

202 Accepted
{
  "id": "run_01J...",
  "state": "queued",
  "agent_version": 3,
  "events_url": "/v1/runs/run_01J.../events"
}
```

Emit SSE events with a numeric monotonic `id` and a typed event name: `run.queued`, `step.started`, `model.delta`, `tool.approval_required`, `tool.completed`, `run.completed`, or `run.failed`. Persist events before publishing; clients use Last-Event-ID to recover after reconnect.

5. Database schema and migration rules

PostgreSQL is the source of truth. Configure a separate application role with only required privileges; migrations run under a short-lived elevated deploy role. All tenant-owned tables carry `tenant_id` `UUID NOT NULL` and a composite index beginning with `tenant_id`.

```
CREATE TABLE runs (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  agent_version_id UUID NOT NULL REFERENCES agent_versions(id),
  state TEXT NOT NULL CHECK (state IN
    ('queued', 'running', 'awaiting_approval', 'retry_scheduled', 'succeeded', 'failed', 'cancelled',
    'timed_out')),
  input_json JSONB NOT NULL,
  output_json JSONB,
  idempotency_key TEXT NOT NULL,
  attempt SMALLINT NOT NULL DEFAULT 0,
  max_steps SMALLINT NOT NULL,
  max_cost_microunits BIGINT NOT NULL,
  spent_cost_microunits BIGINT NOT NULL DEFAULT 0,
  lease_owner TEXT,
  lease_expires_at TIMESTAMPTZ,
  cancel_requested_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, idempotency_key)
);
CREATE INDEX runs_claim_idx ON runs (state, lease_expires_at, created_at)
  WHERE state IN ('queued', 'retry_scheduled');
```

Other required tables are `tenants`, `users`, `api_keys`, `agents`, `agent_versions`, `tool_grants`, `run_steps`, `run_events`, `approval_requests`, `memory_records`, `outbox_events`, `audit_events`, and `provider_health`.

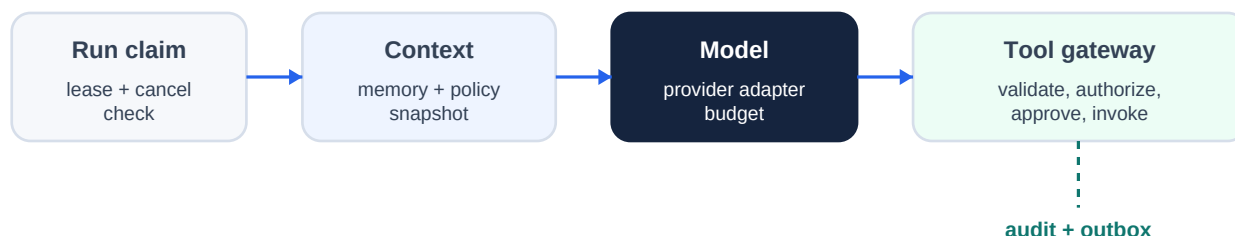
Data and migration policy

- Store raw prompt and tool payloads only when tenant policy permits. Store a redacted projection in `run_steps` for general support access.
- Encrypt sensitive memory content at the application layer with an envelope-encryption key reference; never log plaintext keys.
- Use `pgvector` only after an explicit extension migration. Each vector query must include `tenant_id`, `agent_id`, and an expiration predicate.

- Migrations are forward-only in production. Every destructive change follows expand, backfill, dual-read or dual-write, cutover, then remove in a later release.
- Enforce tenant isolation with PostgreSQL row-level security as defense in depth, and set the tenant id in the database session at request and worker boundaries.

6. Run executor and model provider behavior

Worker execution boundary: persistent state before outbound effect.



Worker algorithm

```

async def execute_run(run_id: UUID, worker_id: str) -> None:
    run = await runs.claim(run_id, worker_id, lease_seconds=60)
    if run.is_terminal:
        return
    await events.append(run_id, "run.started", {})
    context = await context_builder.build(run)
    while run.remaining_steps > 0:
        await cancellation.raise_if_requested(run_id)
        response = await provider_router.complete(context.to_model_request())
        await budgets.record_model_usage(run_id, response.usage)
        await budgets.raise_if_exceeded(run_id)
        if response.final_text:
            await runs.succeed(run_id, redact(response.final_text))
            await outbox.enqueue("run.completed", run_id)
            return
        tool_call = tool_parser.require_single_call(response)
        decision = await policy.authorize(run, tool_call)
        if decision.requires_approval:
            await approvals.create(run, tool_call, decision)
            return
        result = await tool_runner.invoke(run, tool_call, decision)
        context.append_tool_result(result)
        run = await runs.increment_step(run_id)
    await runs.fail(run_id, code="STEP_LIMIT_EXCEEDED")
  
```

Provider adapter requirements

- Normalize provider response, token usage, tool call, retryable error, and safety stop into internal models.
- Pass an explicit request id, tenant id hash, timeout, model name, and trace context to provider calls.
- Set connect timeout to 3 seconds and total provider timeout to 45 seconds by default. All values are configuration, not hard-coded policy.
- Retry only transport reset, bounded 429, and 5xx errors. Use exponential backoff with full jitter and a total attempt budget of 2 retries.
- Record provider error classes and latency in `provider_health`; open a circuit after a configurable failure rate and close only after a probe succeeds.
- Fallback only to an approved model in the same data-residency and capability class. Do not repeat a tool call after a model fallback unless the tool execution is provably idempotent.

7. Memory and retrieval implementation

Memory modes

Mode	Storage	Lifecycle	Production use
ephemeral	In-memory per run plus Redis session cache	TTL 24 hours or shorter	Default for new agents.
session	Redis and encrypted PostgreSQL summary	TTL chosen by tenant	Conversational continuity.
durable	Encrypted PostgreSQL plus pgvector	Retention policy required	Opt-in knowledge and long-lived preferences.

Write pipeline

1. Normalize input to UTF-8 and enforce maximum size.
2. Classify sensitive data and apply tenant-configured redaction before embedding.
3. Chunk to 400 to 800 tokens with 10 percent overlap. Preserve source, namespace, timestamps, and an allowed-use label.
4. Create an embedding through the provider adapter; do not send data to a provider that violates tenant residency rules.
5. Encrypt original content, persist embedding and metadata, enqueue an outbox event, and return a record id.

Retrieval query

Retrieve at most 8 records for v1. Filter by tenant, agent, namespace, expiry, allowed-use label, and optional session id before vector ranking. Include distance and source ids in the model context, but redact raw content if policy demands. Log the record ids used, never the full content, in the run audit trail.

8. Tool security and approval model

Tools are capabilities, not strings. A registry entry contains the input JSON Schema, output schema, side-effect classification, timeout, concurrency limit, allowed network targets, required scopes, and whether human approval is mandatory.

Tool class	Examples	Default policy
Read-only internal	document lookup, retrieval	Allow when granted.
Read-only external	web search, public API read	Allow only through egress allowlist and per-tenant quota.
Effectful reversible	create draft, queue message	Require explicit grant and approval for v1.
Effectful irreversible	send email, write file, execute payment	Deny by default; approval and idempotency key mandatory.
System execution	shell, filesystem, arbitrary browser automation	Not in v1 public runtime; only isolated, reviewed connectors.

Tool gateway sequence

1. Validate tool exists, version is active, and grant belongs to the agent version.
2. Validate arguments against JSON Schema and reject undeclared fields.

3. Evaluate tenant policy, actor scopes, data classification, destination allowlist, cost ceiling, and approval state.
4. Generate a stable execution id and idempotency key. Persist the proposed action before invocation.
5. Execute with a bounded timeout in the required isolation boundary.
6. Persist sanitized result, emit an audit event, and return only the declared output shape to the model.

Never interpolate model output into shell commands, SQL, URLs, file paths, or provider credentials. Treat model-produced identifiers as untrusted input.

9. Reliability, performance, and capacity controls

Limits

Control	Default v1	Enforcement point
HTTP request body	256 KiB	API gateway and FastAPI parser
Run timeout	90 seconds	Worker deadline and provider/tool child deadlines
Model step count	12	Run executor
Tool timeout	20 seconds	Tool gateway
Concurrent runs per tenant	Plan-specific, start at 5	Redis token bucket and worker claim
Queue age alert	60 seconds	Prometheus alert
Event retention	30 days default	Scheduled data-retention job

Apply per-tenant and per-subject rate limits with Redis Lua scripts so check and increment are atomic. Use a global concurrency semaphore for provider-specific traffic. Workers renew leases every 20 seconds; a sweeper requeues expired leases only if the run has no unresolved effectful tool action.

Idempotency and exactly-once effects

API request deduplication is at-least-once safe: duplicate Idempotency-Key requests return the original run. Worker execution is at-least-once. Achieve exactly-once behavior for an effectful integration only through the downstream system's idempotency key and a persisted tool execution record. Do not promise exactly-once execution where the provider cannot support it.

10. Configuration, secrets, and environments

Use typed settings and fail startup for missing required values. Configuration is read once at startup; dynamic controls use a feature-flag service or database-backed policy, never mutable process environment.

```
APP_ENV=production
DATABASE_URL=postgresql+asyncpg://...
REDIS_URL=redis://...
OIDC_ISSUER=https://issuer.example.com/
OIDC_AUDIENCE=aiagentx-api
MODEL_PRIMARY=openai:configured-model
MODEL_FALLBACK=provider:approved-fallback
MODEL_REQUEST_TIMEOUT_SECONDS=45
RUN_TIMEOUT_SECONDS=90
ENCRYPTION_KEY_REF=cloud-kms://key-id
OTEL_EXPORTER_OTLP_ENDPOINT=https://otel-collector.internal:4318
```

- Store secret values in a cloud secret manager and mount or inject them at runtime through workload identity.

- Never commit .env files, API keys, prompt contents, or connection strings.
- Separate development, staging, and production accounts or projects. Production uses isolated databases, Redis, encryption keys, provider credentials, and telemetry destinations.
- Maintain a checked-in .env.example with names and safe example values only.

11. Kubernetes deployment topology

```

Internet -> WAF / API gateway -> api deployment -> PostgreSQL
                                     -> Redis
                                     -> OpenTelemetry collector

Redis queue -> worker deployment -> model providers
                                     -> tool sandbox / approved connectors

CronJob -> retention, expired lease recovery, outbox delivery

```

Deploy separate API and worker workloads. API pods are stateless and do not execute long-running tools. Workers have no inbound public service and use a distinct Kubernetes service account. Create separate deployment manifests for migration jobs, background sweepers, and outbox publishers.

Required platform controls

- TLS termination at gateway and TLS for database, Redis, provider, and telemetry egress.
- Namespace-scoped network policies: only API may receive ingress; worker egress is limited to approved dependencies.
- Read-only root filesystem, non-root UID, dropped Linux capabilities, seccomp profile, resource limits, and admission policy checks.
- Pod disruption budget, horizontal pod autoscaler, topology spread constraints, and a graceful preStop window that drains in-flight work.
- Database backups with point-in-time recovery, restore exercise every quarter, and encryption at rest managed by the platform.

12. Observability, auditing, and operations

Telemetry contract

Every request receives a request_id; every run has a run_id; every step has step_id; every external call is a trace span. Propagate W3C trace headers between API, queue, worker, provider adapter, and tool gateway.

Track: API latency and error rate, queue age, active and failed runs by state, worker lease conflicts, provider latency and error class, tool denials, approval wait time, token and cost usage, memory retrieval count, and redaction count.

Cardinality must be bounded: do not use raw tenant, user, prompt, or run ids as metric labels.

Audit events are append-only and include actor, tenant, action, target, decision, timestamp, trace id, and redacted metadata. Support staff access to sensitive artifacts requires a reason, a time-bound role, and an additional audit event.

SLOs and alerting

- API availability: 99.9 percent monthly, excluding planned maintenance.
- Run admission: 99 percent accepted or rejected with a deterministic validation error within 1 second.
- Queue start: 95 percent of admitted runs begin within 60 seconds at normal capacity.

- Alert on error-budget burn, queue age, provider circuit open, worker lease recovery spikes, failed migrations, audit delivery backlog, and backup failures.

13. Security and privacy review

Perform a threat model before enabling external tools. At minimum address prompt injection, confused deputy behavior, cross-tenant data access, tool credential exfiltration, SSRF, unsafe deserialization, queue replay, model-provider data retention, log leakage, and compromised workload identity.

Threat	Required control
Prompt injection into tool calls	Schema validation, policy gateway, destination allowlists, approval gate.
Cross-tenant access	Auth scopes, tenant filter in every query, RLS, negative integration tests.
SSRF or arbitrary egress	Egress proxy, DNS/IP validation, deny private address ranges, explicit host allowlists.
Secret exposure	Workload identity, secret manager, redaction, no secret values in events.
Replay or duplicate effect	Idempotency key, persisted execution record, downstream deduplication.
Data over-retention	Tenant retention policies, scheduled deletion, deletion evidence, encrypted backups policy.

Require dependency scanning, SAST, secret scanning, container scanning, infrastructure scanning, and a release security review. Document data processing, subprocessors, residency options, and deletion behavior before accepting customer data.

14. Testing, CI/CD, and release plan

Test matrix

Layer	Must cover
Unit	State transitions, budget math, policy decisions, provider error mapping, redaction.
Integration	PostgreSQL RLS, migrations, Redis limits and locks, queue claim and lease recovery.
Contract	OpenAPI responses, provider adapters, tool schemas, SSE resume behavior.
End-to-end	Authenticated run, approval path, cancellation, fallback before effect, retention job.
Load	Queue growth, worker autoscaling, provider throttling, database connection saturation.
Security	Tenant isolation, SSRF rejection, forged approval tokens, secret-redaction assertions.

Pipeline gates

1. Format, lint, type-check, and run fast unit tests on every pull request.
2. Run integration and contract tests against disposable PostgreSQL and Redis services.
3. Build a pinned OCI image, generate an SBOM, sign the image, and scan dependencies and image layers.
4. Apply migrations to staging, run smoke and end-to-end tests, then deploy with canary traffic.
5. Promote only when error rate, latency, queue age, and policy-denial alerts remain within thresholds.
6. Keep a tested rollback plan. Application rollback must be compatible with the current database schema; never couple rollback to a destructive migration.

15. Definition of done for first production release

- Versioned API and OpenAPI contract are published and contract-tested.
- Tenant isolation is enforced in application code and database RLS tests.
- Runs, steps, events, approvals, and audit records are durable and traceable.
- Tool policy is deny-by-default; no arbitrary shell or network egress is available.
- Idempotency, cancellation, timeouts, budgets, retries, and provider circuit breaking are implemented and tested.
- Prompt, tool, and telemetry redaction passes automated negative tests.
- Kubernetes manifests meet platform security baseline; staging restore and rollback drills pass.
- Dashboards, alerts, runbook, incident ownership, backup policy, and data-retention job are in place.
- Load test establishes an explicit initial tenant concurrency and worker-capacity baseline.
- Security, privacy, and operational sign-offs are recorded before customer traffic.

16. Implementation sequence

1. Scaffold the package structure, typed settings, FastAPI health endpoints, CI, and local Docker Compose services.
2. Implement tenant-aware database access, migrations, auth, agent definition CRUD, and API error contract.
3. Implement run creation, idempotency, queueing, worker lease claim, state transitions, and SSE events.
4. Implement one model provider adapter with timeouts, usage accounting, and a deterministic fake provider for tests.
5. Add budget enforcement, cancellation, retry classification, provider health, and circuit breaking.
6. Implement memory modes and retrieval only after tenant isolation and retention controls are tested.
7. Add a single read-only tool through the policy gateway; add approval workflow before any effectful tool.
8. Add observability, audit outbox, deployment manifests, SLOs, load tests, and security review.
9. Run a staged beta with feature flags, measure failure modes, then expand tenant access.